

RankKnot

A 208-Byte In-Memory Rank Summary for Deferred Tensor Analysis

Observe every sample now; choose quantiles, groups, diagnostics, and quantization later

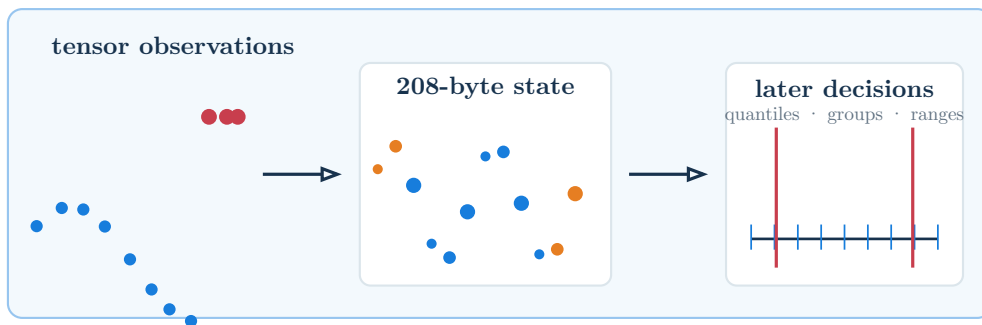
tensor histories

rank transport

exact ties

cold decisions

monatq implementation note · version 0.4



The current implementation RankKnot is now the crate's default statically selected kernel: `TensorDigest<f32>` selects it, and `TensorDigest<f32, TDigest>` opts back out. Thirty-two value knots, 16-bit probability masses, one tie mask, and exact extrema occupy 208 bytes of state per position. The live collector also uses a 256-row input buffer by default and worker-local flush scratch. The generic API includes merge, analysis, zero-filtering, serialization, and optional visualization methods, and RankKnot now implements all of them. Direct support access and quantization remain future work.

An important wording distinction RankKnot incorporates every supported observation that reaches a successful flush, but it does not retain raw values afterward. It preserves an approximate empirical distribution from which later rank-based decisions can be made. Raw samples, their temporal order, and cross-position relationships cannot be reconstructed. NaN input is unsupported and currently fails during flush rather than being rejected atomically by `update`.

Contents

1	Executive summary	2
2	Observe now, decide later	4
2.1	Why fixed summaries are often too early	4
2.2	What cannot be deferred	4
3	A compact empirical distribution	4
3.1	Positive weighted support	4
3.2	The 208-byte K32 summary-state layout	5
4	Streaming compression	5
4.1	Buffer first, compress rarely	5
4.2	Tail-companded rank cells	6
4.3	Mixed cells use weighted means	6
4.4	Prefix-rounded probability	6
5	Cold reconstruction and regrouping	7
5.1	A monotone quantile curve	7
5.2	Choose robust ranges later	7
5.3	Regrouping	7
5.4	Persistence	8
6	Future consumer: post-training quantization	9
6.1	Evaluate an affine quantizer	9
6.2	Search candidates on the stored measure	9
7	Accuracy measurement	10
7.1	Ties require rank intervals	10
7.2	Historical K64 Python feasibility protocol	10
7.3	Historical K64 general-distribution results	10
7.4	Historical K64 discrete observations	11
7.5	Current Rust K32 evidence	11
7.6	Important caveats	11
8	A deliberately small systems design	12
8.1	Rust representation	12
8.2	One shared reducer	12
8.3	Shape classification and zero filtering	12
8.4	Visualization	12
8.5	Default kernel and snapshot identification	13
8.6	Failure reporting	13
8.7	Minimal configuration	13
9	Validation status	13
9.1	Implemented coverage	13
9.2	Open work	14
10	Risks and limitations	14
11	Reproducibility	15
12	Conclusion	16
	References	16

1 Executive summary

`TensorDigest` receives full tensor samples and tracks one distribution at every flat tensor position. `TensorDigest<f32, RankKnot>` is now an implemented, statically selected backend optimized for update throughput and compact state. It

buffers complete samples, compresses each position independently in parallel, and exposes per-position quantiles and exact extrema.

RankKnot separates **observation** from **decision**; today it covers recording, rank queries, cold regrouping, persistence, shape classification, zero filtering, and browser inspection. Generic **TensorDigest** methods provide construction, shape, weight, update, flush, quantile, merge, analysis, zero filtering, serialization, and optional visualization operations, and RankKnot implements the whole contract; it adds sample count, configuration, and tensor/cell extrema on top. Direct knot access and PTQ search remain design directions.



Figure 1: The current recorder. Every supported tensor observation updates the same compact state. Quantiles and extrema are available now; richer cold consumers require future APIs.

The K32 state uses 208 bytes per tensor position: 32 **f32** values, 32 **u16** masses, a 64-bit purity mask, and two **f32** extrema. This is about one twenty-fourth of the repository t-digest’s 4,900-byte summary state and 160 bytes smaller than Quantile Spine’s 368-byte state. Equal values share the ordinary knot budget, and extrema remain exact even though interior observations are compressed.

Memory scope The 208-byte figure is summary state, not complete live memory. With **f32** inputs, the default 256-row buffer adds 1,024 bytes per position, for approximately 1,232 tensor-scaled retained bytes per position. Worker-local vectors and object headers add a small non-tensor-scaled overhead and appear in peak allocator measurements.

Property	t-digest	Quantile Spine	RankKnot K32
Summary state/position	4,900 B	368 B	208 B
f32 input buffer/position	800 B	1,024 B	1,024 B
Additional tensor sort buffer	none	1,024 B	none
Approx. tensor-scaled retained total	5,700 B	2,416 B	1,232 B
State primitive	weighted centroids	rank anchors	up to 32 weighted knots
Repeated values	interpolated	anchor dependent	retained pure interval
Exact min/max	yes	yes	yes
Public <code>merge_*</code> contract	implemented	Unsupported	implemented
Snapshot serialization	implemented	Unsupported	implemented
Analysis and zero filtering	implemented	Unsupported	implemented
Visualization	implemented	Unsupported	implemented
Element types	f32 , i32	f32 , i32	f32 , i32

RankKnot is the default kernel of the crate; the other two columns are opt-in. The table assumes default configurations, **f32**, and a 64-bit target. Allocator instrumentation over 32 positions measured RankKnot at 39,432 live bytes and 45,064 peak bytes, versus 182,408 live bytes for t-digest and 77,320 for Quantile Spine. The component totals explain the scaling; exact allocator totals also include shape vectors, headers, and worker scratch.

Evidence status Current Rust K32 accuracy and memory are measured by `backend_accuracy`, and the repository benchmark exercises update throughput. A local Apple M4 run is reported below. The broader checked-in Python study remains historical K64 feasibility evidence, not a measurement of the current K32 backend. Million-row drift and downstream application quality remain open.

2 Observe now, decide later

2.1 Why fixed summaries are often too early

A minimum and maximum preserve endpoints but reveal nothing about how probability is distributed between them. A fixed histogram commits to bin edges before the eventual query is known. A single percentile commits to one tail policy. A preselected channel grouping prevents later comparison with per-tensor or hardware-aligned alternatives.

RankKnot instead records a reusable approximation of rank mass. The current API supports medians, arbitrary quantiles, percentile bands assembled by the caller, exact per-position extrema, and deferred regrouping. The encoded representation could also support distribution plots, saturation diagnostics, alert thresholds, and PTQ clipping if dedicated consumers or public state access are added.

API status One generic `TensorDigest` contract now includes merge, analysis, zero filtering, serialization, and optional visualization. T-digest and RankKnot both implement all of it; Quantile Spine implements none of it. Every fallible call returns `monatq::Result`, and an unimplemented kernel operation is reported as `Error::Unsupported` rather than as a panic; operations that cannot fail keep infallible signatures. RankKnot state remains crate-private, and callers cannot enumerate knots or hand weighted support directly to a quantizer.

The architecture is closer to a compact tensor history than to a quantizer; today its per-position rank-query, regrouping, and persistence surfaces are implemented.

2.2 What cannot be deferred

No fixed-memory summary retains raw sample identity, temporal order, correlations between positions, or exact arbitrary quantiles. If a later analysis needs sample-level replay, covariance, or causality, it must store different information. Comparison-based quantile sketches necessarily trade memory for error [1]. RankKnot targets representative tensor distributions rather than a universal worst-case guarantee.

3 A compact empirical distribution

3.1 Positive weighted support

An observed stream defines an empirical distribution made of point masses. RankKnot replaces that large measure with at most 32 positive weighted locations. In one dimension, this can be understood as moving nearby rank mass onto representative support points—a small transport coreset [2].

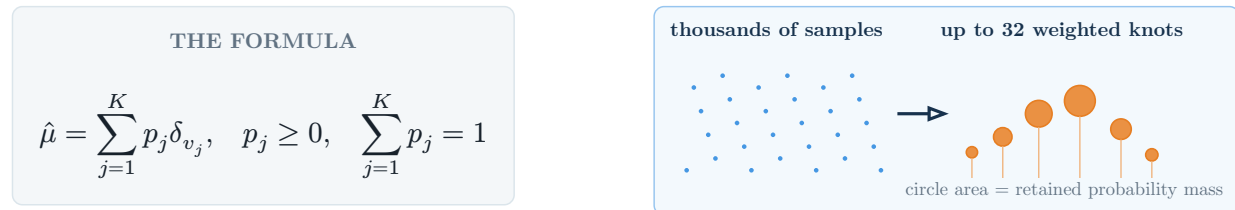


Figure 2: **The compressed history.** Thousands of observations become a short weighted support. Large circles carry more probability; positivity and normalization are explicit.

Every input affects the state, but most inputs cease to be individually identifiable after compression. Any number of distinct values may enter. At most 32 locations remain exact; excess support is approximated by the same rule used for continuous data. There is no separate “number of atoms supported” configuration.

3.2 The 208-byte K32 summary-state layout

All positions receive one observation at the same logical tensor update, so one `u64` sample count lives in RankKnot storage rather than in every position. The per-position state spends its final eight bytes on exact extrema. Input buffering and flush workspace are separate from this layout.

The layout is `f32` for every supported element type. RankKnot accepts `i32` as well as `f32` tensors, but the summary is `f32` in both cases, so an `i32` magnitude above 2^{24} is stored as its nearest representable neighbour. This is a property of the crate rather than of RankKnot: quantiles are reported as `f32` for every element type, and the t-digest kernel keeps `f32` centroids for `i32` tensors too. Only the input buffer and the snapshot dtype tag are element-typed — the former so ingestion stays a `copy_from_slice`, the latter so an `i32` snapshot cannot be decoded as `f32`.

`update` validates one thing and one thing only: that a sample has exactly as many elements as the tensor. A wrong length is rejected with `Error::ShapeMismatch` before anything is buffered, so the digest is unchanged. NaN is **not** validated. It is a documented precondition, deliberately unchecked to keep the ingestion loop free of a per-element scan, and a NaN reaches the buffer and panics in the flush sort comparator. Ingestion is therefore atomic with respect to sample length but not with respect to NaN.

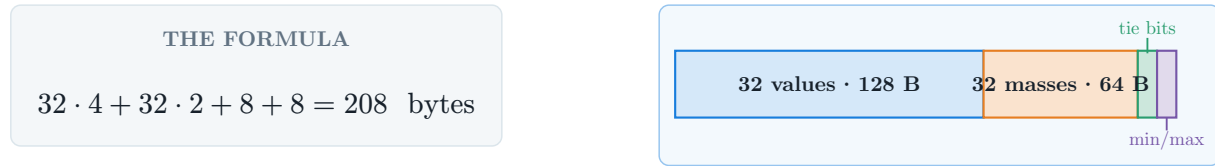


Figure 3: **Summary-state accounting.** The four terms are `f32` locations, `u16` masses, a 64-bit tie mask, and two `f32` extrema. Zero mass identifies an unused slot.

Field	Bytes	Meaning
<code>values[32]: f32</code>	128	mixed-group means or retained exact values
<code>masses[32]: u16</code>	64	probability quanta; active masses sum to 65,535
<code>pure_mask: u64</code>	8	one bit per active location; upper 32 bits are unused
<code>min, max: f32</code>	8	exact endpoints, including supported infinities
Summary-state total	208	no pointers and no per-position allocation

Per-position missingness would break the shared-count assumption and must pay an explicit memory or semantic cost. Atomic NaN rejection would require an ingestion validation pass; the current performance-oriented contract instead assumes NaN-free observations. Positive and negative infinity are supported and protected as pure singleton groups.

4 Streaming compression

4.1 Buffer first, compress rarely

Updates append full row-major tensor samples to one input buffer whose public `buffer_capacity` defaults to 256. A flush handles positions independently using Rayon, with a minimum parallel chunk length of 64:

1. Gather one position into a worker-local `Vec<f32>` and unstable-sort it with `partial_cmp`.
2. Linearly merge it with up to 32 old representatives, coalescing equal values during the merge.
3. Evaluate 31 fixed arcsine cuts and snap each to the nearest entry boundary.
4. Skip duplicate or unusable cuts; fewer than 32 groups may remain. Protect infinities as pure singleton groups.
5. Store exact pure singletons and `f64`-accumulated weighted means, rounded to `f32`, for mixed groups.
6. Prefix-round cumulative masses to 65,535 with ties-to-even, difference adjacent prefixes, and update extrema.

At the default capacity the merged stream contains at most 288 entries. Scratch consists of two preallocated worker-local vectors and one fixed 31-element boundary array. It scales with parallel work rather than tensor width; there is no `numel × batch_rows` sorted copy, per-position hash table, or iterative optimizer. The vectors do allocate when a Rayon job initializes, so “no dynamic allocation” applies only to persistent per-position state, not flush workspace.

4.2 Tail-companded rank cells

Uniform rank cells spend the same resolution at the median and at the extremes. Many later decisions—robust ranges, alarms, saturation estimates, and quantization—care about tails. RankKnot therefore places fixed slots uniformly in an abstract coordinate that bends toward probability zero and one.

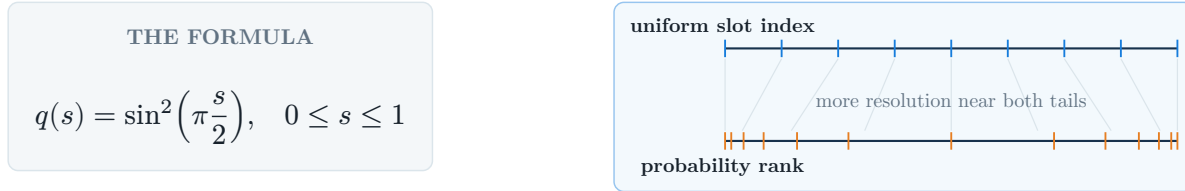


Figure 4: **Tail-aware rank placement.** Equal slot steps on the upper line become denser at both ends of probability space. This is the same broad tail-allocation principle that benefits t-digest [3].

A large exact tie may contain several desired cuts. Duplicate snapped cuts are skipped, so a 50% zero mass consumes one support location rather than half the state. The current implementation does not recursively refill unused cuts; a tie-heavy stream can therefore finish with fewer than 32 active locations.

4.3 Mixed cells use weighted means

A cell containing several locations stores their weighted mean. It is the constant representative minimizing squared value reconstruction error, and masses plus first moments combine cleanly before the next lossy repartition.

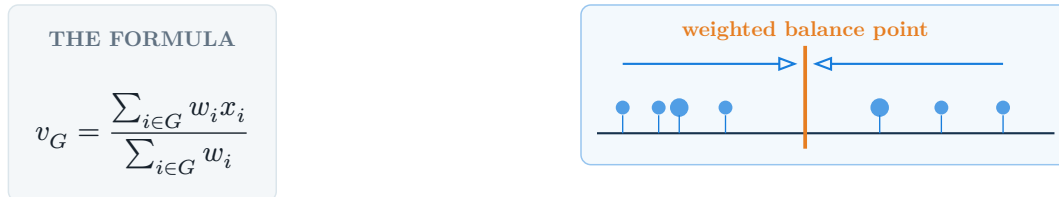


Figure 5: **One representative per mixed cell.** The orange line is the weighted balance point. In the feasibility prototype, medians produced materially larger smooth-distribution error under cold interpolation.

A cell containing one exact location retains that location and sets its purity bit. A mixed cell remains mixed even if its rounded mean happens to equal an input value, preventing accidental invention of a tie.

4.4 Prefix-rounded probability

Sixteen-bit probability masses keep the K32 state compact. The exact total sample count remains outside each position. Cumulative prefixes are rounded first, then differenced, so every represented CDF boundary is directly controlled.

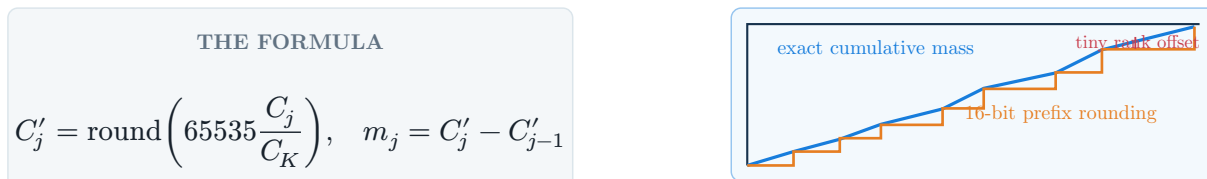


Figure 6: **Sixteen-bit mass encoding.** Each orange step follows the blue cumulative mass. Prefix rounding avoids independent mass errors all drifting in one direction.

The current boundary discrepancy is at most one half probability quantum. Repeated compression and requantization can still accumulate approximation; long streams and repeated distribution shifts remain mandatory tests.

5 Cold reconstruction and regrouping

5.1 A monotone quantile curve

Each mixed knot is anchored at the center rank of its encoded mass. Neighboring mixed anchors are joined linearly. A pure knot contributes a horizontal interval across its mass. Sorting values and ranks makes the resulting quantile curve monotone.

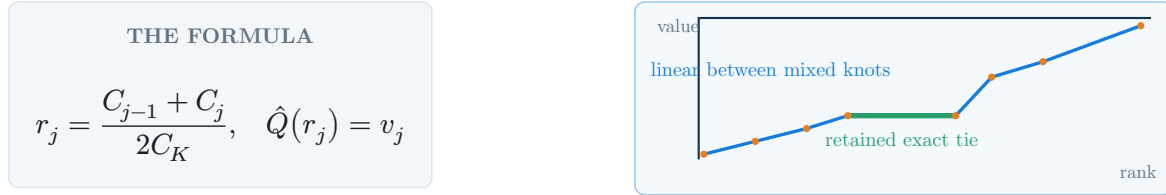


Figure 7: **Decode only when needed.** Blue segments interpolate unresolved continuous mass. The green plateau represents an exact retained tie across a whole rank interval.

Endpoint requests bypass this interpolation. For a nonempty summary, any $q \leq 0$ returns exact `min`, any $q \geq 1$ returns exact `max`, and a NaN probability returns NaN. An empty summary returns `0.0` for every probability, including NaN, because the emptiness check runs first. Queries flush pending rows first and can therefore trigger the unsupported-input panic if buffered observations contain NaN. Tensor-wide queries run in parallel; `cell_quantiles` evaluates one position locally. When an infinity is adjacent to a finite anchor, the interpolation helper returns the infinity throughout that gap rather than producing NaN.

5.2 Choose robust ranges later

A caller can defer its tail policy until after collection. One consumer may want a central visualization band; another may choose an anomaly threshold; another may estimate a clipping interval. All can use different tail probabilities from the same state.



Figure 8: **A deferred central range.** The shaded total tail mass is selected later. Collection does not hardcode τ , symmetry, or the eventual use of the range.

5.3 Regrouping

The generic contract exposes `merge_cells`, `merge_channels`, and `merge_all`, and RankKnot implements all three. Each stored position is treated as a positive weighted measure: its active knots are emitted as weighted support, the union is sorted once, equal values are coalesced (a merged knot stays pure only if every contributor was pure), and the same arcsine compressor used by ingestion produces the single output state. Extrema are exact unions of the source extrema, so endpoint queries on a merged digest remain exact.

Because the sample count is shared by every position in a storage, all sources carry identical total mass and the 16-bit masses can be unioned unscaled; the general form below reduces to the uniform case. Working unscaled also keeps the merged total far from `u64` saturation. The merged digest reports a sample count of $n \cdot |G|$ and can continue to accept updates, which weight old mass correctly against new rows. `merge_channels` treats a channel as a contiguous block of $H \times W$ positions and, unlike the t-digest kernel, compresses the union in a single pass: knot weights are `u64` and moments accumulate in `f64`, so a second lossy repartition would buy no precision. Merging an empty selection yields an empty single-position digest.

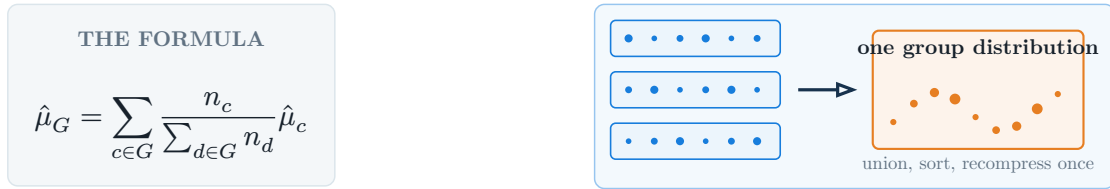


Figure 9: **Deferred grouping.** Fine-grained histories become one group distribution without replaying the original tensor samples. Grouping policy is a cold decision.

Aggregation latency is not a performance requirement, so the cold merge allocates temporary vectors and globally sorts all input support rather than reusing the worker-local flush scratch.

Merging recompresses already-compressed support, so it is intuitive to expect it to be worse than compressing the pooled stream once. Measurement does not support that intuition. Over 999 probabilities against the **exact** pooled population, merged mean rank error ranged from 0.30x to 1.53x the error of a reference digest fed the pooled stream directly, and was lower on six of eight tested channel layouts. Separated modes benefit most: each source resolves its own mode with its own 32 knots before the union is recompressed, whereas a single digest sees the modes interleaved and must spend its budget on both at once. That comparison lives in `merged_distribution_tracks_the_exact_pooled_population`, which asserts per-workload caps at roughly 2x headroom and additionally requires merged mean error to stay within 2x of the pooled reference.

`backend_accuracy` reports merge fidelity as its own table, merging all 32 positions of each representative workload into one digest and scoring it against the exact pooled population. Quantile Spine is skipped rather than called, and its cells are drawn as a struck-through rule.

Merged workload	K32 mean / max	t-digest mean / max	Spine
normal	0.000426 / 0.001220	0.000753 / 0.001970	n/a
uniform	0.000656 / 0.000897	0.001645 / 0.005659	n/a
lognormal	0.000859 / 0.001984	0.004540 / 0.020050	n/a
exponential	0.000846 / 0.001493	0.003785 / 0.015527	n/a
laplace	0.000691 / 0.001500	0.001599 / 0.005166	n/a
overlapping-bimodal	0.000428 / 0.001177	0.001373 / 0.005538	n/a
32-level-normal	0.000137 / 0.000618	0.009275 / 0.030787	n/a
50%-zeros	0.000362 / 0.001130	0.033803 / 0.250106	n/a
95%-zero-activations	0.000240 / 0.002045	0.000418 / 0.002980	n/a
heterogeneous-tensor	0.000978 / 0.002736	0.007044 / 0.026593	n/a

Unlike the per-position table, where no backend wins everywhere, K32 had both lower mean and lower maximum merged rank error than t-digest on all ten workloads. The merged digest retains 1,240 bytes against t-digest’s 5,708, and the merge itself peaks at 17,880 transient bytes against t-digest’s roughly 38,000: the union of 32 states is one `Vec` of at most 1,024 entries, sorted and coalesced in place. These figures cover single merges of one storage; repeated merge-of-merge chains are not yet measured.

5.4 Persistence

Deferred analysis across process boundaries requires a snapshot format, and `to_bytes`, `from_bytes`, and `from_payload` now provide one. A snapshot flushes pending rows and stores the shape, the shared sample count, and the per-position knots as flat parallel vectors, bincode-encoded and zstd-compressed.

The encoding constants travel with the data. `RANK_KNOT_K`, the mass quanta, a format version, and a kernel tag are all recorded and checked on load, because those constants are documented as unstable internals: a future K64 build must reject a K32 file rather than decode masses that no longer mean what it assumes. The kernel tag also occupies the leading payload byte that the crate-level loader reads as a t-digest dtype tag, so handing a RankKnot snapshot to the t-digest reader is a clean error instead of a misplay. Decoded states are validated before use: active masses must normalize, knot values must ascend, and NaN is rejected.

`buffer_capacity` is deliberately **not** persisted. It is an ingestion tuning knob that does not affect the encoded distribution, so a loaded digest starts from the default capacity and a caller who tuned it must reapply that choice. A loaded digest is otherwise a fully live collector and can continue to accept updates, weighting restored mass against new rows by the restored sample count.

A 1,024-position `f32` digest serialized to 165,914 bytes, or 162.0 bytes per position against 208 bytes of live state, since `zstd` exploits the many inactive knot slots. The corresponding `t-digest` snapshot was 991,701 bytes, or 968.5 bytes per position.

6 Future consumer: post-training quantization

PTQ illustrates why deferred distribution storage could be useful. A calibration run could collect activation histories before choosing bit width, symmetric versus asymmetric ranges, per-tensor versus per-channel grouping, or clipping policy. No current RankKnot API exposes weighted knots to a quantizer, and no quantizer consumes this state; the following equations describe a proposed consumer. PTQ methods commonly use a small representative set to establish activation ranges [4], [5].

6.1 Evaluate an affine quantizer

A bounded real interval determines the step spacing of a finite integer codebook [6], [7]. Wider endpoints preserve outliers but make every interior step coarser.

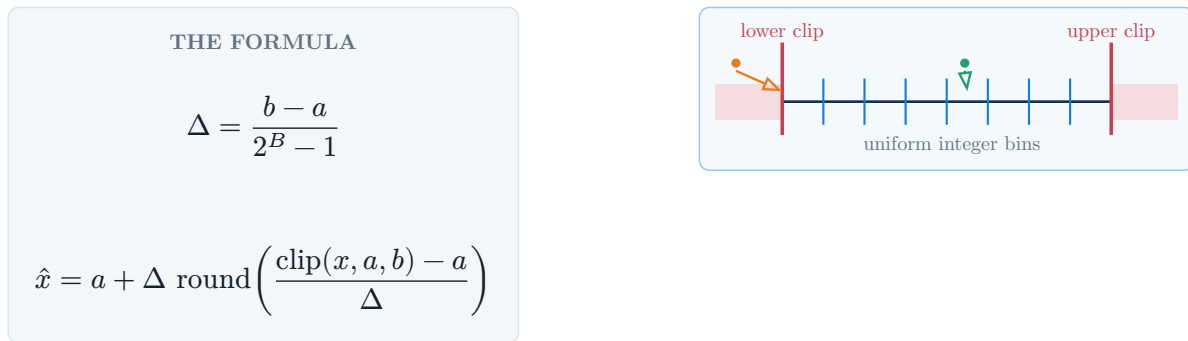


Figure 10: **One possible deferred quantizer.** The red endpoints and bit width are not chosen while observations are collected. They are candidate decisions evaluated afterward.

6.2 Search candidates on the stored measure

With a future support-export or dedicated PTQ API, a cold quantizer could evaluate candidate intervals directly on up to 32 weighted locations. This would approximate activation reconstruction error while making search cost independent of the original sample count.

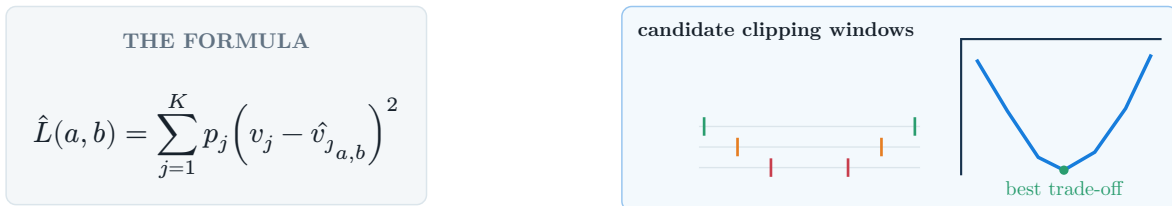


Figure 11: **Quantization is a consumer, not the state.** Narrow, medium, and wide candidate windows trade clipping against rounding. Other consumers can ignore this objective entirely.

This marginal objective does not model Hessians, cross-channel correlations, or final task loss. More sophisticated quantization methods can consume the same distributional evidence alongside their own model-specific information.

7 Accuracy measurement

7.1 Ties require rank intervals

A requested quantile inside an empirical jump is correctly answered by the repeated value. Measuring only one side of the CDF would report an error even though the returned value is a valid generalized quantile [8].

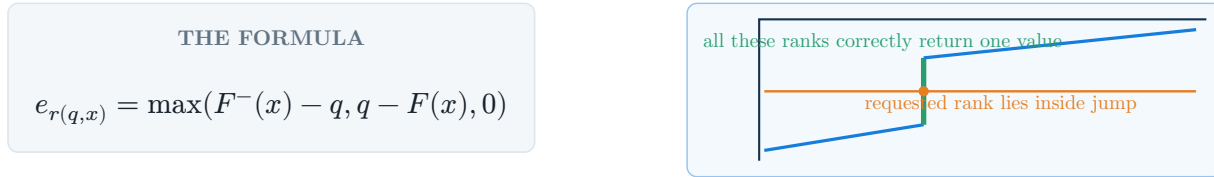


Figure 12: **Tie-aware rank error.** The orange requested rank lies inside the green CDF jump, so returning the repeated value has zero error.

Application-specific consumers must report their own outcomes as well. Rank error evaluates the recorder; it does not replace downstream quality, reconstruction error, alert precision, or visualization usefulness.

7.2 Historical K64 Python feasibility protocol

The checked-in historical experiment uses two held-out seeds, 32,768 f32 observations per workload, 256-row K64 prototype flushes, and 235 query probabilities from one ten-thousandth through its upper complement. Parameter exploration used a separate seed. The Python prototype simulates a **64-knot** state with 16-bit prefix masses and persistent f32 representatives. Repository **TDigest** and **QuantileSpine** run through a temporary Rust crate on identical ordered values using their own default buffering. These measurements do not describe the current K32 Rust implementation.

Workloads include uniform, normal, two lognormal scales, Student-t, shuffled and blocked mixtures, one and five added atoms, 50% zero, constants, 8–256-level quantization, and identical uniform multisets in random, ascending, and descending order. Ordered cases are diagnostics rather than proposed gates.

7.3 Historical K64 general-distribution results

Workload	K64 mean	t-digest mean	K64 max	t-digest max
uniform	0.000267	0.001414	0.001150	0.005179
normal	0.000271	0.001018	0.001069	0.005746
lognormal, $\sigma=1$	0.000346	0.002420	0.001304	0.008954
lognormal, $\sigma=2$	0.000584	0.004453	0.001637	0.011169
Student-t, df=2	0.000392	0.000861	0.001288	0.004347
bimodal, shuffled	0.000927	0.009599	0.013983	0.066852
bimodal, blocked	0.001288	0.010447	0.016979	0.066882

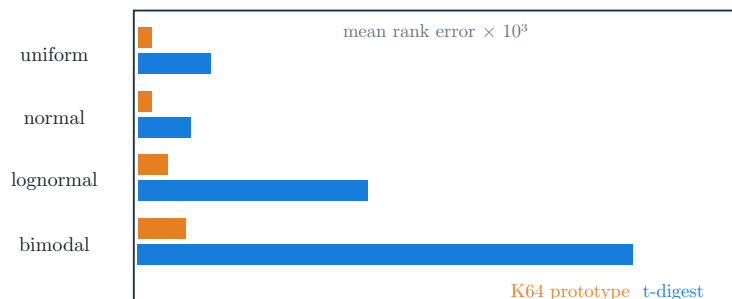


Figure 13: Historical K64 Python mean valid-rank error on selected held-out workloads. Bars use thousandths of rank. Lower is better; separated modes remain harder than smooth distributions.

Across all 36 workload/seed pairs, the historical K64 prototype’s mean-of-means was 0.000330 and its worst observed maximum was 0.016979. The repository t-digest values were 0.006928 and 0.246068. K64 had lower mean and maximum

error on every nonconstant pair; both methods tied on constants. This is capacity-design evidence, not a claim about current K32.

7.4 Historical K64 discrete observations

Workload	Mean error	Worst error	Interpretation
five added atoms	0.000117	0.001653	ties share the ordinary knot budget
8 levels	0	0	all levels retained
16 levels	0	0	all levels retained
32 levels	0	0	all levels retained
64 levels	0	0	all levels retained in this test
256 levels	0.000831	0.002344	excess levels interpolated

Zero means zero at the 235 tested ranks, not equality at every possible probability. The historical result demonstrates that repeated values need no separately hardcoded atom capacity.

7.5 Current Rust K32 evidence

The current `backend_accuracy` executable uses 100,000 samples at each of 32 representative tensor positions, nine query probabilities from 0.001 through 0.999, and tie-aware empirical rank intervals. RankKnot and Quantile Spine use their default 256-row buffers; t-digest uses its default 200-row buffer. Its adversarial suite uses 65,536 samples at one position and 1,003 probabilities. Heap figures come from an instrumented global allocator and exclude inputs, exact truth, and query outputs.

Workload	K32 mean / max	t-digest mean / max	Spine mean / max
normal	0.000458 / 0.001460	0.000627 / 0.006950	0.000302 / 0.001490
uniform	0.000681 / 0.001170	0.001334 / 0.007380	0.000575 / 0.001780
lognormal	0.000860 / 0.002290	0.001831 / 0.012570	0.000758 / 0.002130
32-level normal	0.000094 / 0.000660	0.009511 / 0.034780	0.011590 / 0.055840
50% zeros	0.000380 / 0.001380	0.026949 / 0.252720	0.000096 / 0.000600
95% zeros	0.000508 / 0.006160	0.000359 / 0.004750	0.000031 / 0.000420
heterogeneous	0.000511 / 0.002260	0.003170 / 0.051750	0.001411 / 0.039770

K32 substantially improves on t-digest for most representative workloads, especially quantized and 50%-zero data, but it is not uniformly best. Quantile Spine has lower mean error on several smooth distributions and clearly wins on 95%-zero activations. K32 often has the smallest maximum error and is particularly strong on the heterogeneous workload. Adversarial results are mixed, so no universal ordering is claimed.

Allocator measurements at 32 positions were 39,432 live / 45,064 peak bytes for K32, 182,408 / typically 239,208 for t-digest, and 77,320 / 113,160 for Quantile Spine. At one position K32 measured 1,240 live / 6,872 peak bytes; the difference between live and peak is worker-local ingestion scratch.

A local Apple M4 Divan run with RankKnot’s default capacity 256 and each comparison backend’s default configuration measured median update times of 8.73 ms for K32 versus 7.72 ms for t-digest on $64 \times 64 \times 1,000$ normal data, 8.81 versus 7.51 ms on the corresponding uniform data, and 21.47 versus 26.81 ms on $256 \times 256 \times 200$ uniform data. K32 therefore remained 13–17% slower on the smaller repeated-flush cases but was about 20% faster on the larger one. These are platform-specific observations, not portable guarantees.

7.6 Important caveats

- **Historical extreme tails:** in the K64 study, t-digest was often two to three times more accurate at individual smooth tail queries, although K64’s absolute tail errors were small.
- **Separated modes:** linear interpolation across a large unsupported value gap remains a principal failure mode.
- **Extrema:** current K32 endpoint requests use exact min/max sidecars; the historical K64 interior results did not include endpoint requests.

- **Throughput scope:** the Apple M4 timings above are local medians. Different tensor widths, Rayon pools, CPUs, and system load can change the comparison.
- **No consumer evidence:** no RankKnot quantizer, alert, or visualization study exists because those APIs are not implemented. Snapshots round-trip correctly, but no application has yet been built on a reloaded digest.

8 A deliberately small systems design

8.1 Rust representation

```
const RANK_KNOT_K: usize = 32;
```

```
#[repr(C)]
struct RankKnotState {
    values: [f32; RANK_KNOT_K],
    masses: [u16; RANK_KNOT_K],
    pure_mask: u64,
    min: f32,
    max: f32,
}
```

`RankKnotState` is crate-private; the public `RankKnot` type is a zero-sized kernel marker. One `u64` sample count is shared across positions in `RankKnotStorage`. Temporary weights and weighted moments use `u64` and `f64`; persistent representatives round to `f32`. With 256 buffered rows, tensor-scaled retained storage is approximately $208 + 1,024 = 1,232$ bytes per position. Worker scratch and object headers are additional.

8.2 One shared reducer

The deterministic boundary/compression reducer is shared by buffered ingestion and by the cold `merge_*` path; only the way weighted support is assembled differs. Current RankKnot callers cannot access the encoded support, and no RankKnot-specific cache or query-time workspace is retained per position.

8.3 Shape classification and zero filtering

`analyze` is not kernel-specific work. The procedure standardizes 50 probe quantiles by the summary's own median and interquantile spread, then picks the nearest fixed reference profile in L1; the only thing it needs from a kernel is a rank query. It is therefore implemented once, against a quantile closure, and both t-digest and RankKnot pass their own evaluator into it. A degenerate position, with no measurable spread or no observations at all, is reported as `Unknown` rather than forced into a family.

`without_zeros` drops knots sitting at zero and re-runs the survivors through the same compressor used by ingestion, which renormalizes their masses back to full scale. The result answers quantiles of the nonzero subpopulation. One consequence deserves stating: RankKnot's sample count is storage-wide rather than per position, and the number of nonzero observations differs per position, so the filtered digest carries the original count unchanged and that count no longer agrees with the filtered distribution. A filtered digest is a shape to inspect, not a population to count.

This exposed a latent precondition in the compressor. Normalizing a running prefix onto the mass axis was computed as `cumulative / (total / MASS_QUANTA)`, which is correct only when the total weight is an exact multiple of one quantum. Ingestion and merge both satisfy that, because every contribution carries a full quantum; a filtered subset does not, and the divisor collapsed to zero. The normalization now multiplies before dividing, in 128-bit arithmetic so the product cannot overflow. Ingestion and merge results are bit-identical either way.

8.4 Visualization

The HTTP visualizer never needed kernel-specific access: every endpoint is expressible in terms of merge, analysis, zero filtering, and cell quantiles. It is now generic over kernel storage, so implementing the contract is sufficient to be servable and no second copy of the server exists. Analysis runs before the listener binds, so a kernel that cannot classify fails immediately rather than serving a window that errors on every request, and a per-request failure becomes a 500 response instead of tearing down the server.

8.5 Default kernel and snapshot identification

RankKnot is the default kernel: `TensorDigest<T, K>` declares `K = RankKnot`, so `TensorDigest<f32>` selects it while every explicit `TensorDigest<f32, TDigest>` continues to compile unchanged. Defaulting a type parameter is additive, which is what makes the switch possible without a breaking change on the Rust side.

Snapshots are self-describing. RankKnot writes a kernel tag ahead of everything else, and the crate-level loader deserializes only that prefix to decide how to proceed before decoding any knots; t-digest snapshots, which lead with a bare dtype tag and carry no marker of their own, are the fallback. `monatq::from_bytes` and `monatq::load` therefore return an `AnyTensorDigest` naming both the kernel and the element type. Detection is deliberately confined to the untyped entry points: asking `TensorDigest<f32, TDigest>` directly to load a RankKnot snapshot remains an error, because convenience at the untyped boundary is not a licence to reinterpret one kernel's state as another's.

The Python bindings cannot carry a Rust type parameter, so the static choice is reified into a four-arm enum over (element type, kernel) with macro-generated dispatch; each arm remains a monomorphised digest. `kernel="rankknot"` is the constructor default. Tuning knobs are kernel-specific and validated: passing `compression` to RankKnot or `buffer_capacity` to t-digest raises rather than being silently discarded, on the grounds that an ignored accuracy knob surfaces much later as an unexplained result.

8.6 Failure reporting

Because kernels are selected statically but do not all implement the same operations, the contract has to express “this kernel cannot do that” without aborting the caller. Every fallible call returns `monatq::Result`; a missing kernel operation is `Error::Unsupported`, carrying the kernel and operation names. Tooling that sweeps all backends, such as `backend_accuracy`, uses this to skip a kernel and draw a placeholder rather than crash the run, and the Python bindings translate it into `NotImplementedError` instead of letting a Rust panic cross the FFI boundary.

Operations that cannot fail keep infallible signatures, so a quantile query needs no error handling. Snapshot problems are `Error::InvalidSnapshot`, kept distinct from `Error::Io`: the former means the bytes were read successfully but do not describe a digest this build can trust, which is what a version, kernel, or invariant mismatch actually is.

8.7 Minimal configuration

Item	Current value	Status
<code>buffer_capacity</code>	256	public and configurable; must be positive
input type	<code>f32</code> , <code>i32</code>	summary state is <code>f32</code> for both; <code>i32</code> above 2^{24} rounds
knot count	32	fixed internal constant
mass quanta	65,535	fixed internal <code>u16</code> normalization
rank scale	arcsine	fixed internal targets
NaN	unsupported	documented precondition; unchecked, panics during flush
sample length	checked	<code>Error::ShapeMismatch</code> ; rejected before buffering
infinities	supported	protected pure singleton groups and exact endpoints

Only `buffer_capacity` is public configuration. The mass encoding, knot count, and scale function remain implementation details while long-stream and cross-platform behavior stabilizes.

9 Validation status

9.1 Implemented coverage

Area	Current evidence
Buffering	default/custom capacity, partial and full flushes, and sample count

Area	Current evidence
Queries	monotone curves, NaN query probability, exact endpoints, and tensor/cell paths
Discrete data	retained tie intervals, constants, and deterministic signed-zero extrema
Exceptional values	protected positive and negative infinities
Encoding	ordered support, active masses summing to 65,535, and ties-to-even quantization
Merging	cell/channel/tensor unions, extrema, empty selections, post-merge updates, and tie-aware rank error against exact pooled truth on eight workloads
Persistence	byte and file round trips, empty digests, signed zero and infinities, post-load updates, cross-kernel rejection, truncation, and each snapshot guard
Accuracy/memory	<code>backend_accuracy</code> representative, tensor-wide merge, and adversarial suites with allocator instrumentation
Throughput	Divan benchmarks for normal and uniform tensors at two representative sizes
Error contract	rejected sample lengths leave the digest untouched, out-of-bounds positions, unsupported-kernel reporting, and a <code>Display</code> that does not repeat its own source
Default kernel	all four kernel/dtype snapshot combinations autodetected by the untyped loader; typed loaders still reject a foreign kernel; Python kernel selection, per-kernel config validation, and default-kernel round trips
Element types	i32 ingestion, merge, and snapshot round trip; dtype-tag rejection of a cross-type load; a pinned test for the 2^{24} f32 precision ceiling, asserting where exactness holds and where it stops
Analysis	each reference family recovered from 32 knots; <code>Unknown</code> for constant and unobserved positions; independent per-position classification
Zero filtering	interquartile range recovered from an 80%-zero spike; nonzero extrema preserved; all-zero positions yield an empty summary; filtered state survives a snapshot round trip

9.2 Open work

- revisit the NaN precondition if a caller ever needs validated ingestion; it is currently a deliberate documented contract rather than an open question;
- test requantization drift through at least one million updates and repeated shifts;
- check in reproducible current K32 accuracy and benchmark result artifacts with platform metadata;
- implement the Quantile Spine operations, which report `Unsupported` across the whole contract, and add fidelity tests;
- calibrate the `analyze` unknown-threshold against RankKnot's 32-knot curve rather than inheriting the t-digest figure, and measure how often near-boundary shapes fall back to `Unknown`;
- measure repeated merge-of-merge chains, which the current single-merge fidelity table does not cover;
- add RankKnot state export, so a cold consumer can read weighted knots directly rather than only through quantile queries;
- build visualization, PTQ, or threshold consumers and measure application outcomes;
- continue throughput work on smaller repeated-flush tensors, where K32 still trails t-digest locally.

Ascending streams, blocked modes, and crafted float patterns remain useful diagnostics rather than universal gates. Fixed memory does not provide a worst-case distribution-free accuracy guarantee.

10 Risks and limitations

- **Approximation, not archival:** a snapshot preserves the compressed summary, but individual observations and their temporal order are irrecoverable from it.
- **Representative data:** no summary can infer a deployment regime that was never observed.

- **Marginal state:** cross-position covariance and sample identity are absent.
- **Requantization drift:** 16-bit masses compactly encode 32 locations but repeatedly approximate old probability.
- **Shared count and NaN:** NaN-free input is a precondition rather than a checked invariant. NaN reaches the buffer and panics during flush; this is the one remaining panic on the ingestion path, deliberately kept off the hot loop. Per-position missingness would require revised semantics.
- **Tail and mode parity:** no backend wins every workload; sparse zero activations, extreme tails, and separated modes remain important counterexamples.
- **Working memory:** the 208-byte title names state only. The default input buffer raises tensor-scaled retained storage to about 1,232 bytes per f32 position, and worker scratch raises peak heap.
- **Unavailable consumers:** knots remain crate-private; PTQ and direct weighted-support export are absent, so a quantizer still cannot read the summary without going through rank queries.
- **Classification on 32 knots:** analyze standardizes 50 probe quantiles against fixed reference profiles. RankKnot answers those probes from at most 32 knots rather than a t-digest's 100 centroids, so family separation rests on a coarser curve. The reference families are separable in testing, but near-boundary or mixture shapes are more likely to fall back to Unknown.
- **Unproven format stability:** the snapshot format is versioned and validated, but it has only ever been written and read by one build. Cross-version and cross-platform round trips are untested.
- **Merge is still lossy:** a single merge measured no worse than one-pass pooled compression on the tested workloads, but it does recompress approximate support. Repeated merge-of-merge chains are unmeasured and are the expected place for drift to appear.
- **Fixed memory:** no 208-byte state offers tiny distribution-free error for every stream.

The design principle Collection should preserve a useful approximation of the observed distribution while making as few downstream choices as possible. RankKnot spends its state budget on rank mass, ties, and extrema; everything else is deferred.

11 Reproducibility

Current K32 Rust behavior is defined and exercised by:

- `monatq/src/kernels/rankknot.rs`
- `monatq/tests/rankknot.rs`
- `monatq/tests/error_shape.rs`
- `monatq/tests/rankknot_analysis.rs`
- `monatq/src/error.rs`
- `monatq/src/bin/backend_accuracy.rs`
- `monatq/benches/tensor_digest.rs`

Run `cargo run -p monatq --release --bin backend_accuracy` for the current accuracy/memory report, which prints per-position, tensor-wide merge, and adversarial tables, and `cargo bench -p monatq --bench tensor_digest` for throughput. The numeric K32 results in this note came from a local Apple M4 run and are not yet checked in as a machine-readable artifact.

Historical K64 feasibility evidence remains in:

- `docs/experiments/transport_coresset_feasibility.py`
- `docs/experiments/results/transport_coresset_holdout.csv`
- `docs/experiments/README.md`

That script simulates 48-location u32 and 64-location u16 layouts and invokes repository backends through a temporary Rust crate. Its unqualified “RankKnot” label means the historical K64 prototype, not current K32.

12 Conclusion

Current RankKnot is a working K32, f32-only quantile backend. Its 208-byte per-position state preserves exact extrema and retained pure ties, its default live allocation is substantially smaller than both comparison backends, and current Rust accuracy is generally stronger than t-digest on the representative suite without being uniformly best. Local throughput beats t-digest on the larger tested tensor and still trails it on smaller repeated-flush cases.

The broader K64 Python results remain useful evidence for the design’s capacity trade-off, but they are not current-backend results. Cold regrouping and persistence are now implemented through `merge_cells`, `merge_channels`, `merge_all`, and the versioned snapshot format; analysis, zero filtering, and visualization are now implemented too, and direct knot access, clipping search, and quantization are absent. The next decisions should be driven by long-stream drift, reproducible cross-platform throughput, an explicit NaN contract, and real consumer studies rather than by the historical prototype alone.

References

- [1] Z. Karnin, K. Lang, and E. Liberty, “Optimal Quantile Approximation in Streams,” in *2016 IEEE 57th Annual Symposium on Foundations of Computer Science*, 2016, pp. 71–78. doi: [10.1109/FOCS.2016.17](https://doi.org/10.1109/FOCS.2016.17).
- [2] G. Peyré and M. Cuturi, *Computational Optimal Transport*, vol. 11, no. 5–6. Now Publishers, 2019. doi: [10.1561/22000000073](https://doi.org/10.1561/22000000073).
- [3] T. Dunning and O. Ertl, “Computing Extremely Accurate Quantiles Using t-Digests,” *arXiv preprint arXiv:1902.04023*, 2019, doi: [10.48550/arXiv.1902.04023](https://doi.org/10.48550/arXiv.1902.04023).
- [4] I. Hubara, Y. Nahshan, Y. Hanani, R. Banner, and D. Soudry, “Accurate Post Training Quantization With Small Calibration Sets,” in *Proceedings of the 38th International Conference on Machine Learning*, in Proceedings of Machine Learning Research, vol. 139. 2021, pp. 4466–4475. [Online]. Available: <https://proceedings.mlr.press/v139/hubara21a.html>
- [5] M. Nagel, M. Fournarakis, R. A. Amjad, Y. Bondarenko, M. van Baalen, and T. Blankevoort, “A White Paper on Neural Network Quantization,” *arXiv preprint arXiv:2106.08295*, 2021, doi: [10.48550/arXiv.2106.08295](https://doi.org/10.48550/arXiv.2106.08295).
- [6] R. M. Gray and D. L. Neuhoff, “Quantization,” *IEEE Transactions on Information Theory*, vol. 44, no. 6, pp. 2325–2383, 1998, doi: [10.1109/18.720541](https://doi.org/10.1109/18.720541).
- [7] B. Jacob *et al.*, “Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, pp. 2704–2713. [Online]. Available: https://openaccess.thecvf.com/content_cvpr_2018/html/Jacob_Quantization_and_Training_CVPR_2018_paper.html
- [8] R. J. Hyndman and Y. Fan, “Sample Quantiles in Statistical Packages,” *The American Statistician*, vol. 50, no. 4, pp. 361–365, 1996, doi: [10.1080/00031305.1996.10473566](https://doi.org/10.1080/00031305.1996.10473566).